

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА
ІМЕНІ ЮРІЯ КОНДРАТЮКА»

Навчально-науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

Лабораторна робота № 2

з навчальної дисципліни

"Технології розробки корпоративних Web-додатків"

Виконав:

Студент 402 – ТН

Марков Іван Андрійович

Перевірив:

Тютюнник Петро Богданович

Полтава 2026

Лабораторна робота №2

Проектування архітектури програмного забезпечення та моделювання даних

Завдання

1. Обґрунтування вибору технологічного стеку (Technology Stack) На основі вимог, зібраних у першій лабораторній роботі, оберіть технології для реалізації проєкту.

- Створіть документ або розділ у звіті, де чітко визначте:
 - Мова програмування Back-end: (наприклад, C#/.NET, Java/Spring, Node.js, Python/Django).
 - Фреймворк Front-end: (наприклад, React, Angular, Vue.js або Server-Side Rendering — Razor/Thymeleaf).
 - Система керування базами даних (СКБД): (PostgreSQL, MS SQL, MongoDB, Redis тощо).
 - Інструменти DevOps/Hosting: (Docker, GitHub Actions, Azure/AWS — опціонально).
- Надайте обґрунтування: Чому саме ці технології підходять для вирішення вашої бізнес-задачі? (Наприклад: "Обрано MongoDB, оскільки структура даних каталогу товарів є гнучкою та часто змінюється" або "Обрано .NET через надійну типізацію та швидкість розробки корпоративних API").

2. Архітектурне моделювання за методологією C4 (C4 Model) Спроектуйте архітектуру вашої системи, створивши три рівні діаграм C4. Це допоможе зрозуміти структуру системи від загального до конкретного.

- Рівень 1: System Context Diagram (Контекст).
 - Покажіть вашу систему як одну "чорну скриньку" в центрі.
 - Відобразіть усіх користувачів (Actors) та зовнішні системи (наприклад, платіжні шлюзи, API поштових розсилок), з якими вона взаємодіє.
- Рівень 2: Container Diagram (Контейнери).
 - "Відкрийте" систему і покажіть її основні технічні блоки (контейнери).

- Це можуть бути: Single Page Application, Mobile App, API Application, Database, File System.
- Вкажіть технології для кожного контейнера та протоколи взаємодії між ними (HTTP/HTTPS, TCP, GRPC).
- Рівень 3: Component Diagram (Компоненти).
 - Деталізуйте один основний контейнер (наприклад, API Application).
 - Покажіть внутрішні компоненти (наприклад: AuthController, EmailService, UserRepository) та їхні зв'язки.

3. Проектування бази даних (ER-Model / Data Schema) Спроектуйте схему зберігання даних. Вибір типу діаграми залежить від обраної СКБД:

- Варіант А: Реляційна база даних (SQL)
 - Побудуйте класичну ER-діаграму (Entity-Relationship Diagram).
 - Вкажіть сутності, атрибути, первинні (PK) та зовнішні (FK) ключі.
 - Визначте типи зв'язків (1:1, 1:N, M:N).
 - Забезпечте нормалізацію даних (мінімум до 3-ї нормальної форми).
- Варіант Б: Нереляційна база даних (NoSQL)
 - Створіть діаграму або схему колекцій/документів.
 - Відобразіть структуру JSON-документів для основних сутностей.
 - Вкажіть стратегію зв'язків: Embedding (вкладення даних) чи Referencing (посилання на ID).
 - Обґрунтуйте вибір стратегії (наприклад, "Використано Embedding для коментарів, щоб отримати пост разом із коментарями за один запит").

Очікуваний результат (Звіт):

1. Таблиця або список обраного тех стеку з обґрунтуванням (1-2 абзаци).
2. Три діаграми C4 (Context, Container, Component) з коротким описом до кожної.
3. Схема бази даних (ER-діаграма або схема документів NoSQL).

Хід роботи

Завдання №1.

1.1 Таблиця технологічного стеку.

Шар системи	Технологія
Back - end	C#/.NET
Front - end	React
СКБД	SQLite

1.2 Обґрунтування вибору.

Back - end: C#/.NET

Обрано C# та ASP.NET Core через:

- вбудовану підтримку REST;
- автоматичну систему Dependency Injection;
- інтеграцію зі Swagger;
- простоту реалізації шарової архітектури;
- підтримку Entity Framework Core.

Front - end: React

React обрано тому що:

- забезпечує швидкий рендеринг інтерфейсу;
- дозволяє легко створювати адаптивний UI;
- добре підходить для роліової моделі (читач, бібліотекар, адміністратор).

СКБД: SQLite

Обрано SQLite через:

- простоту розгортання;
- відсутність необхідності встановлення серверу БД;
- зберігання у вигляді одного файлу;
- достатню продуктивність для бібліотечної системи.

Завдання №2.

2.1 System Context Diagram

Актори:

- Читач - надсилає SimpleLibrary API запити на пошук, бронювання, перегляд історії;
- Бібліотекар - надсилає SimpleLibrary API запити на видачу/повернення книг;
- Адміністратор - надсилає SimpleLibrary API запити на керування користувачів.

SimpleLibrary API у відповідь на запити повертає акторам результати операції.

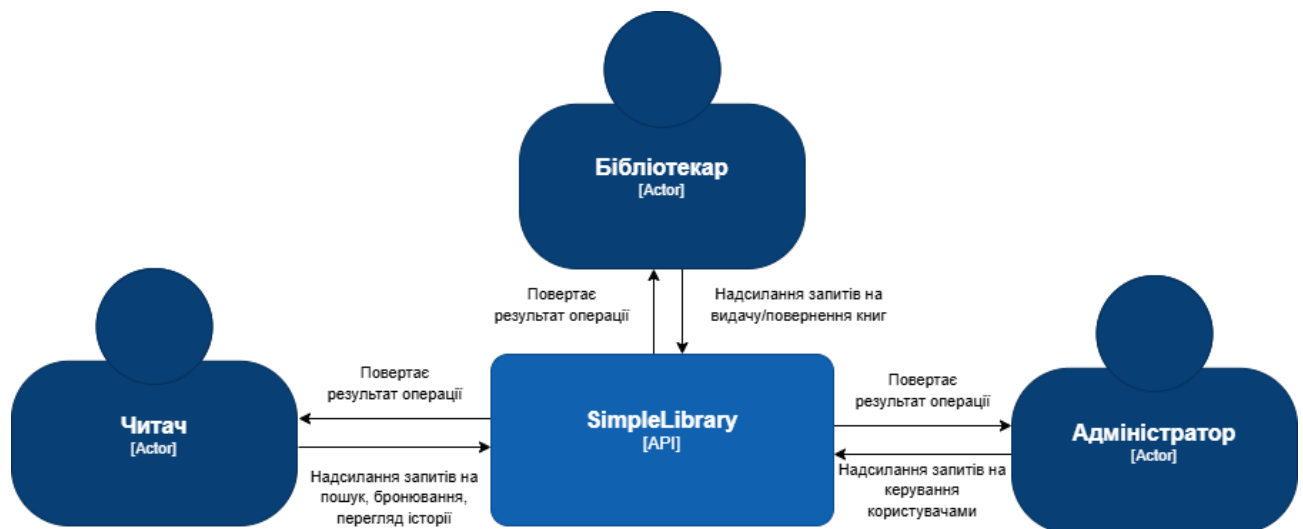


Рисунок 1 - System Context Diagram, зроблена через draw.io.

2.2 Container Diagram

Таблиця контейнерів.

Контейнер	Технологія	Призначення
Web App	Chrome / Firefox / Edge	Відображає інтерфейс і надсилає запити HTTP до Web API.
Web API	ASP.NET Core	Обробка HTTP - запитів. Аутентифікація та авторизація. Реалізація бізнес-логіки. Робота з базою даних. Повернення JSON - відповідей.
DB	SQLite	Зберігання користувачів. Зберігання книг. Зберігання видач. Зберігання бронювань.

Взаємодія контейнерів.

1. Web App - Web API

Протокол: HTTPS

Формат: JSON

Надсилає:

- GET /books
- POST /loans
- POST /auth/login

Отримує:

- JSON - відповіді;

- HTTP status codes (200, 404, 400).

2. Web API - DB

Протокол: SQL (через Entity Framework Core)

Надсилає:

- SELECT
- INSERT
- UPDATE
- DELETE

Отримує:

- Результати запитів;
- Підтвердження операцій.

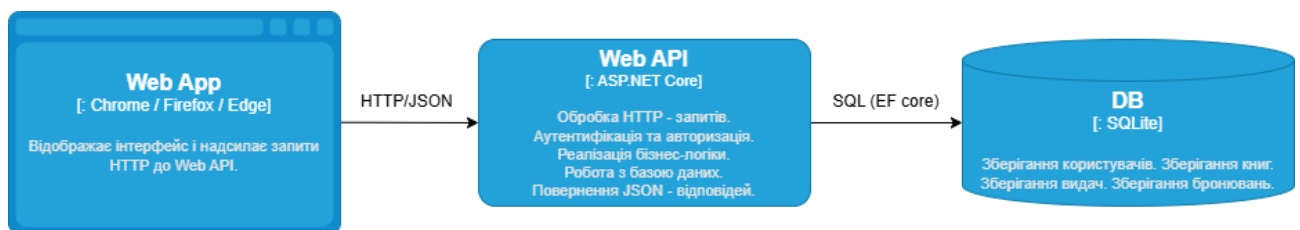


Рисунок 2 - Container Diagram, зроблена через draw.io.

2.3 Component Diagram

Основні компоненти

1. Controllers (API Layer)

Відповідають за:

- Прийом HTTP - запитів
- Валідацію вхідних даних
- Виклик сервісів
- Повернення DTO

Компоненти:

- AuthController

- UsersController
- LoansController
- BooksController

2. Service Layer (Business Logic)

Інкапсулює бізнес - правила:

- AuthService
- UserService
- LoanService
- BookService

Приклад бізнес - логіки:

- Перевірка доступності книги
- Зменшення available_copies
- Перевірка терміну повернення
- Перевірка ролі користувача

3. Repository Layer (Data Access)

Працює із БД через EF Core.

- UserRepository
- BookRepository
- LoanRepository

Завдання:

- CRUD операції
- Робота з DbContext

4. Database Context

- ApplicationDbContext
- Успадковується від DbContext
- Містить DbSet <User>, DbSet <Book> і т.д.

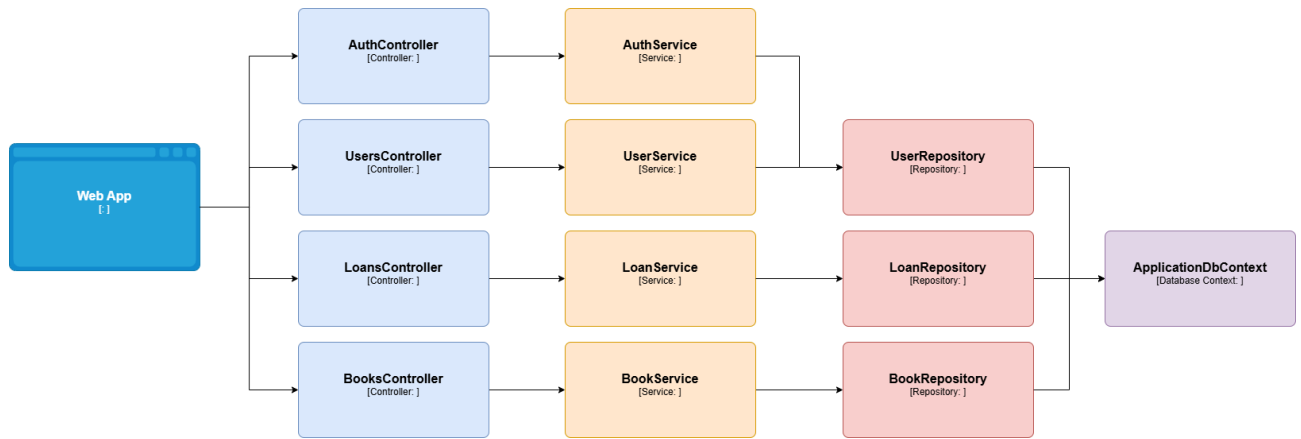


Рисунок 3 - Component Diagram, зроблена через draw.io.

Завдання №3

Опис структури бази даних SimpleLibrary

1. Таблиця Roles

Таблиця призначена для зберігання ролей користувачів у системі.

Поля:

- id (PK, integer, increment) - первинний ключ таблиці. Унікальний ідентифікатор ролі.
- name (varchar(50), unique, not null) - назва ролі (Reader, Librarian, Admin).
Забезпечує розмежування доступу відповідно до рольової моделі.

2. Таблиця Users

Таблиця містить інформацію про зареєстрованих користувачів системи.

Поля:

- id (PK, integer, increment) - унікальний ідентифікатор користувача.
- full_name (varchar(100), not null) - повне ім'я користувача для відображення у системі.
- email (varchar(100), unique, not null) - електронна пошта користувача.
Використовується для авторизації та є унікальною.
- password_hash (varchar(255), not null) - хеш пароля користувача.
Зберігається у зашифрованому вигляді для забезпечення безпеки.

- role_id (FK > Roles.id, not null) - зовнішній ключ, що визначає роль користувача в системі.
- created_at (datetime, not null) - дата та час реєстрації користувача.

3. Таблиця Books

Таблиця містить інформацію про книги бібліотечного фонду.

Поля:

- id (PK, integer, increment) - унікальний ідентифікатор книги.
- title (varchar(200), not null) - назва книги.
- author (varchar(150), not null) - автор книги.
- genre (varchar(100)) - жанр книги (може бути NULL).
- isbn (varchar(20), unique) - міжнародний стандартний номер книги. Є унікальним.
- total_copies (integer, not null) - загальна кількість примірників книги в бібліотеці.
- available_copies (integer, not null) - кількість доступних для видачі примірників.

4. Таблиця Loans

Таблиця зберігає інформацію про видачу книг користувачам.

Поля:

- id (PK, integer, increment) - унікальний ідентифікатор операції видачі.
- user_id (FK > Users.id, not null) - ідентифікатор користувача, який отримав книгу.
- book_id (FK > Books.id, not null) - ідентифікатор виданої книги.
- issued_at (datetime, not null) - дата та час видачі книги.
- due_date (datetime, not null) - дата, до якої книгу необхідно повернути.
- returned_at (datetime) - дата фактичного повернення книги. Якщо NULL - книга ще не повернена.

5. Таблиця Reservations

Таблиця призначена для зберігання інформації про бронювання книг.

Поля:

- id (PK, integer, increment) - унікальний ідентифікатор бронювання.
- user_id (FK > Users.id, not null) - ідентифікатор користувача, який здійснив бронювання.
- book_id (FK > Books.id, not null) - ідентифікатор книги, яку заброньовано.
- reserved_at (datetime, not null) - дата та час створення бронювання.
- status (varchar(50), not null) - статус бронювання (Active, Cancelled, Completed).

Опис зв'язків між таблицями

- Roles (1) - (N) Users
- Users (1) - (N) Loans
- Books (1) - (N) Loans
- Users (1) - (N) Reservations
- Books (1) - (N) Reservations

Відповідність 3-й нормальній формі (3NF)

- Усі таблиці мають первинний ключ.
- Усі неключові атрибути залежать лише від первинного ключа.
- Відсутні транзитивні залежності.
- Дані не дублюються.

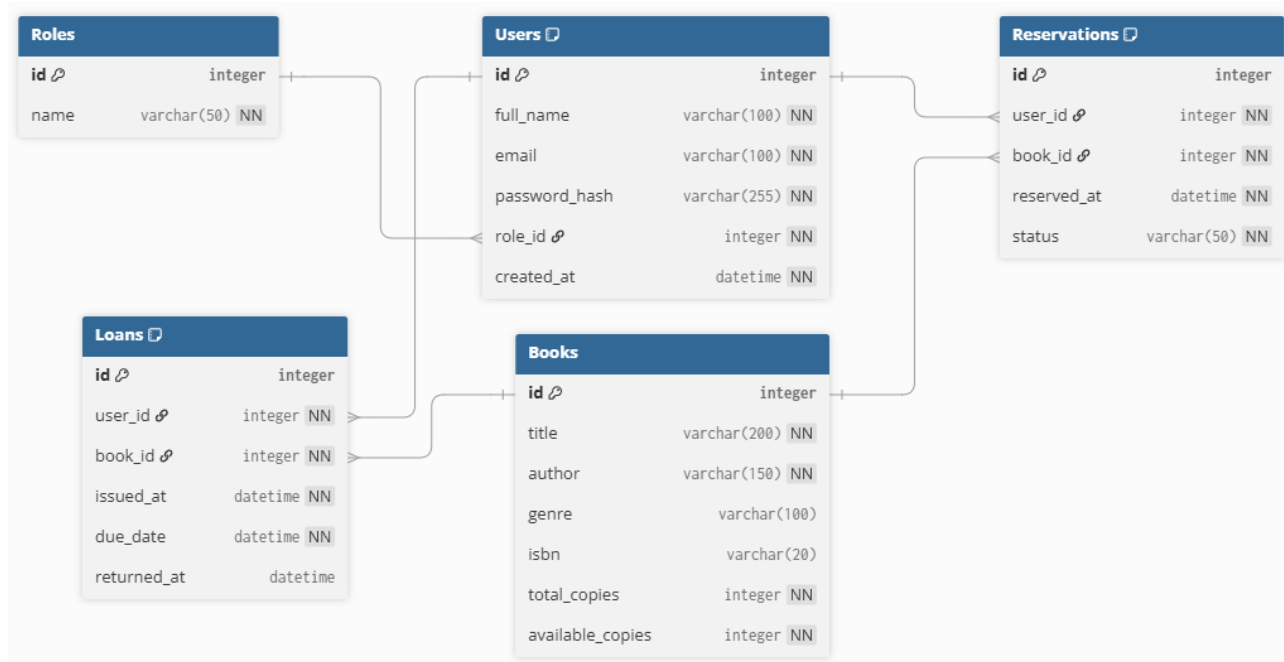


Рисунок 4 – ER - діаграма бази даних, зроблена через dbdiagram.io.